Urban Mobility Explorer: Technical Report

Authors: Albertine Umuhoza (Backend), Eunice Nina Sangwa (Database), Kevine Niyonkuru (Frontend), Karen Stephy Musangwa (Algorithms & Documentation) **Dataset:** NYC TLC Yellow Taxi Trip Records, January 2019 **Repository:** https://github.com/tinnah13/Urban-Mobility-Data

---

1. Executive Summary

Urban Mobility Explorer is an end-to-end analytics platform for ~7.7 million NYC yellow-taxi trips. We built a reproducible pipeline that cleans the raw data, enriches it with derived features, stores it in a normalized SQLite star schema, serves it through a Flask REST API, and presents it in an interactive dashboard. Core ranking and outlier logic is implemented with custom algorithms rather than library routines.

Headline results

- **99.2%** of raw rows survived cleaning (7,635,365 of 7,696,617); **7,463,010** trips were loaded into the database.
- **Demand peaks at 18:00** (500,639 trips), roughly **16×** the 04:00 low.
- **~92%** of pickups occur in Manhattan.
- **Airport trips average $49.52** versus **$13.16** for city trips, a **3.8×** premium.

---

2. Problem Statement and Goals

Raw TLC data is large, noisy and unlabeled by geography. Our goals were to:

1. Turn it into a trustworthy, queryable dataset with a documented cleaning process.
2. Design a relational model that supports fast aggregate queries.
3. Expose the data through a clean API and a dashboard a non-technical user can explore.
4. Demonstrate algorithmic understanding by implementing sorting, top-K selection and outlier detection by hand.

---

3. Data

Source	Description
`yellow_tripdata_2019-01.parquet`	Fact data: 18 columns, ~7.7M trips (105 MB parquet)
`taxi_zone_lookup.csv`	265 zone IDs mapped to borough and service zone
`taxi_zones/` shapefile	Spatial metadata for zones

We chose **parquet over CSV**: the file is ~6× smaller, loads in ~3 s versus ~60 s, and avoids pandas dtype-inference failures seen on the CSV.

---

4. Methodology

4.1 Pipeline overview

```
Parquet/CSV → loader → cleaner (6 rules) → features (8) →
                              |
                        outlier_log → docs/cleaning_log.md
```

4.2 Cleaning rules and results

Six rules are applied in `cleaner.py`. Every exclusion is counted and logged for transparency.

Rule	Rows removed
Duplicate records	0
Missing critical fields	0
Temporal anomalies (e.g. dropoff before pickup, dates outside Jan 2019)	6,562
Distance anomalies (zero/negative or physically implausible)	48,975
Fare anomalies (zero/negative or extreme)	5,714
Passenger-count anomalies	1
Total removed	**61,252**

Raw 7,696,617 → Kept 7,635,365 (99.2%). Distance anomalies were the largest category (80% of all removals).

During database loading a further **172,355** trips were skipped because they referenced the TLC "Unknown" zones (IDs 264/265), which are intentionally excluded from the zone dimension. Final table size: **7,463,010 trips** (97.0% of raw).

4.3 Feature engineering

Eight derived features are added in `features.py` before loading, including **trip speed**, **tip percentage** and an **airport flag**, plus further

time and fare-based fields used by the dashboard and views. These make the downstream queries (hourly demand, airport comparison, speed analysis) simple and index-friendly.

4.4 Database design

A **star schema** keeps dimensions small and the fact table narrow to join.

Table	Rows	Role
`boroughs`	7	Dimension
`service_zones`	5	Dimension
`taxi_zones`	263	Dimension (FK → borough, service zone)
`payment_types`	6	Lookup
`rate_codes`	7	Lookup
`dim_time`	2,468,631	Time dimension (unique timestamps)
`trips`	7,463,010	Fact table (24 columns, 13 indexes)

Two views pre-package common analytics: `v_hourly_demand` and `v_zone_performance`. Indexes target the filter columns used by the dashboard (borough, payment type, fare, airport flag, pickup hour).

Trips are inserted in **batches of 100,000** inside transactions, which keeps memory bounded and makes the ~15-minute build resumable and debuggable.

4.5 Backend API

A Flask application (`backend/app.py`) with an environment-driven config exposes **11 JSON endpoints** (health, summary KPIs, hourly demand, boroughs, top zones, filtered trips, anomalies, filter metadata, insights, ping, version). It also serves the static frontend so the whole system runs from a single process on port 5000. Business logic lives in `services/`, keeping routes thin.

4.6 Frontend dashboard

Plain HTML/CSS/JavaScript with Chart.js, structured as `api.js` (fetch wrapper), `charts.js` (renderers) and `app.js` (controller). The dashboard shows 7 KPI cards, 4 charts, a filter bar, a 100-row sample table, and a story panel with three insights. Filters re-query the API rather than filtering in the browser, so the UI never needs to load millions of rows.

---

5. Custom Algorithms

The assignment constraint was to avoid built-in sorting and heap utilities (`sorted()`, `heapq`, `numpy`, `scipy`).

5.1 Merge sort

```
MERGE_SORT(A):
    if len(A) <= 1: return A
    mid ← len(A) / 2
    L ← MERGE_SORT(A[0:mid])
    R ← MERGE_SORT(A[mid:])
    return MERGE(L, R)

MERGE(L, R):
    out ← []; i ← 0; j ← 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: append L[i]; i++     # <= keeps
        else:            append R[j]; j++
    append remaining of L and R
    return out
```

- **Time:** O(n log n) in best, average and worst case. **Space:** O(n).
- **Why:** guaranteed worst case (unlike quicksort) and stability, so equal keys keep their original order.

5.2 Bounded top-K min-heap

```
TOP_K(items, K):
    heap ← empty min-heap (size ≤ K)
    for x in items:
        if size(heap) < K:          push(heap, x)
        else if x > heap.min:       replace_min(heap, x)
    return MERGE_SORT(heap contents, descending)
```

- **Time:** O(n log K). **Space:** O(K).
- **Why:** for "top 10 zones of 263", K ≪ n, so we avoid sorting everything. The heap never grows beyond K elements. Used by `/api/top-zones`.

5.3 IQR anomaly detector

```
IQR_OUTLIERS(values):
    S  ← MERGE_SORT(values)
    Q1 ← percentile(S, 25);  Q3 ← percentile(S, 75)
    IQR ← Q3 − Q1
    lower ← Q1 − 1.5 × IQR;  upper ← Q3 + 1.5 × IQR
    return { v in values : v < lower or v > upper }
```

- **Time:** O(n log n), dominated by the sort. **Space:** O(n).
- **Why IQR instead of mean ± 3σ:** taxi fares and distances are heavily right-skewed with extreme values. Mean and standard deviation are themselves distorted by those outliers, while quartiles are not. Used by `/api/anomalies`.

---

6. Findings

6.1 Demand follows a clear daily rhythm

18:00 is the busiest hour with **500,639 trips**, about **16×** the quietest hour (04:00). Demand rises through the morning, plateaus midday, and peaks in the evening commute window. Implication: fleet supply and driver incentives matter most in the late afternoon and evening.

6.2 Manhattan dominates pickups

Roughly **92%** of pickups originate in Manhattan. The outer boroughs share the remaining ~8%, so their service coverage is comparatively thin. Implication: outer-borough demand is small in volume but potentially underserved.

6.3 Airport trips carry a large fare premium

Airport trips average **$49.52** versus **$13.16** for in-city trips (**3.8×**). Longer distances and flat/negotiated airport rates drive the gap. Implication: airport pickups are disproportionately valuable per trip.

---

7. Challenges and Lessons Learned

Challenge	Resolution
CSV loading failed on mixed dtypes (`Number is not int`)	Standardized on parquet
Python 3.14 incompatible with `pyarrow<20`	Pinned supported range to Python 3.10–3.13
`FOREIGN KEY constraint failed` during load	Excluded unknown zones 264/265 deliberately and counted the skipped rows
15-minute ETL runtime	Batched inserts (100k) and progress logging; documented rebuild steps

Large result sets slowing the UI	Server-side filtering/aggregation and a 100-row sample table
`git push` blocked on some networks	Documented SSH / hotspot workaround

---

8. Limitations and Future Work

- **Single month:** results reflect January 2019 only; seasonality cannot be assessed. Loading additional months into the same schema is straightforward.
- **Yellow taxis only:** green cabs and for-hire vehicles are excluded, so Manhattan's 92% share overstates its share of *all* NYC trips.
- **SQLite:** ideal for a self-contained academic project, but PostgreSQL would be better for concurrent users and spatial queries (PostGIS).
- **Test coverage:** `backend/tests/` is scaffolding; unit tests for the algorithms and API contract tests are the next priority.
- **Spatial visualization:** the zone shapefile is loaded but not yet used for a choropleth map.
- **Further ideas:** weather and event overlays, fare prediction, caching for expensive endpoints, containerization with Docker.

---

9. Team Contributions

Member	Responsibilities
Albertine Umuhoza	Flask API, endpoint design, services layer, configuration
Eunice Nina Sangwa	Star schema, indexes, views, ETL orchestration (`build_database.py`, `seed.py`)

Kevine Niyonkuru	Dashboard layout, Chart.js visualizations, filter bar, API client
Karen Stephy Musangwa	Merge sort, top-K heap, IQR detector, cleaning log, README and this report

Detailed per-member evidence is in `codevibe-reports/`.

---

10. Reproducibility

```
git clone https://github.com/tinnah13/Urban-Mobility-Data
cd Urban-Mobility-Data
python -m venv .venv && source .venv/bin/activate     # W
pip install -r backend/requirements.txt
# place yellow_tripdata_2019-01.parquet in data/
python backend/db/build_database.py                  # ~1
python backend/app.py                                # th
```

11. References

- NYC Taxi & Limousine Commission, *TLC Trip Record Data*: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
- NYC TLC, *Trip Record User Guide*
- NYC TLC, *Taxi Zone Lookup Table*
- Chart.js, Flask and SQLAlchemy documentation